

UNIT III HTML,CSS CLIENT SIDE SCRIPTING 9

Understanding Internet - Understanding websites and web servers - HTML - – forms – frames – tables –web page design – CSS - Java Script: An introduction to JavaScript–JavaScript DOM Model-Date and Objects,-Regular Expressions-JavaScript Events - Event Handling and Validation -Exception Handling

Understanding the Internet

The Internet is a vast network of servers and computers which are connected to one another via phone lines, microwave, satellites, etc. Servers are simply computers that store large amounts of information whose purpose is to “serve” the information they contain to other computers.

The Internet has often been described as “The Information Superhighway.” Generally, smaller networks such as the systems found in schools and universities, hospitals, government offices and private business, etc., are all interconnected to form a larger network known as the Internet.

The Information contained within the Internet can be retrieved and displayed in a variety of mediums such as telnet, gopher and, most recently, the World Wide Web (WWW) or “The Web.” With telnet, the user (computer operator) must know complex commands in order to access the desired information. Gopher is an easier method to retrieve information where the user types in key words in order to search a database (computer storage of information). The easiest, and by far the most popular, method to retrieve information is through the World Wide Web. The World Wide Web is made up of information that is displayed as “Web pages” or html documents (html stands for hypertext markup language). These Web pages contain hypertext which allows the user to immediately access other Web pages by selecting underlined words or phrases. Most recently this technique has become more sophisticated to allow the user to simply “click” the computer mouse on pictures or icons to access other Web pages. The software that allows the user to click on icons is known as a graphical interface or “browser.” The first successful browser was Mosaic and now the most popular browser is Netscape Navigator.

web page

A document which can be displayed in a web browser such as Firefox, Google Chrome, Opera, Microsoft Edge, or Apple Safari. These are also often called just "pages."

website

A collection of web pages which are grouped together and usually connected together in various ways. Often called a "website" or a "site."

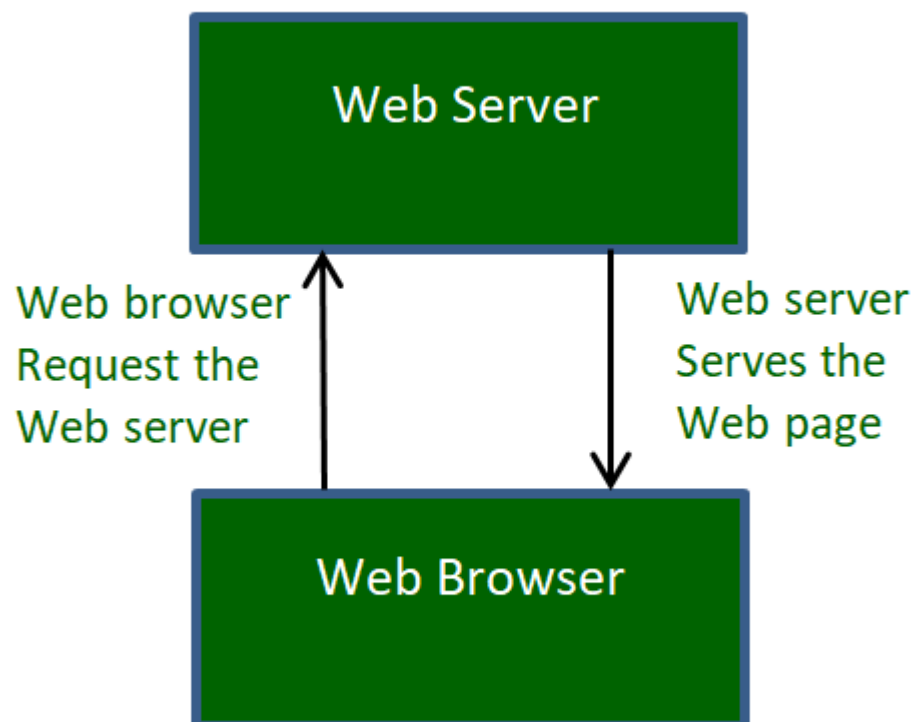
web server

A computer that hosts a website on the Internet.

A web server is a computer that runs websites. It's a computer program that distributes web pages as they are requisitioned. The basic objective of the web server is to store, process and deliver web pages to the users.

How do Web servers work?

A page on the internet can be viewed, when the browser requests it from the web server and the web server responds with that page. A simple diagrammatic representation of this is given below in the figure:



The simple process consists of 4 steps, they are:

- **Obtaining the IP Address from the domain name:** Our web browser first obtains the IP address the domain name (for e.g., for this page the domain

name is www.geeksforgeeks.org) resolves to. It can obtain the IP address in 2 ways-

- By searching in its cache.
- By requesting one or more DNS (Domain Name System) Servers.
- **Browser requests the full URL:** After knowing the IP Address, the browser now demands a full URL from the web server.
- **The web server responds to request:** The web server responds to the browser by sending the desired pages, and in case, the pages do not exist or some other error occurs, it will send the appropriate error message.

For example:

You may have seen **Error 404**, while trying to open a webpage, which is the message sent by the server when the page does not exist.

Another common one is **Error 401** when access is denied to us due to incorrect credentials, like username or password, provided by us.

- **The browser displays the web page:** The Browser finally gets the webpages and displays it, or displays the error message.

What is HTML?

HTML stands for Hypertext Markup Language, and it is the most widely used language to write Web Pages.

Hypertext refers to the way in which Web pages (HTML documents) are linked together. Thus, the link available on a webpage is called Hypertext.

HTML is a Markup Language which means you use HTML to simply "mark-up" a text document with tags that tell a Web browser how to structure it to display.

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

HTML Tags

HTML tags are like keywords which defines that how web browser will format and display the content. With the help of tags, a web browser can distinguish between an HTML content and a simple content. HTML tags contain three main

parts: opening tag, content and closing tag. But some HTML tags are unclosed tags.

When a web browser reads an HTML document, browser reads it from top to bottom and left to right. HTML tags are used to create HTML documents and render their properties. Each HTML tags have different properties.

An HTML file must have some essential tags so that web browser can differentiate between a simple text and HTML text. You can use as many tags you want as per your code requirement.

- All HTML tags must enclosed within < > these brackets.
- Every tag in HTML perform different tasks.
- If you have used an open tag <tag>, then you must use a close tag </tag> (except some tags)
- The <!DOCTYPE html> declaration defines that this document is an HTML5 document

The declaration tag is used by the web browser to understand the version of the HTML used in the document. Current version of HTML is 5 and it makes use of the following declaration:

- The <html> element is the root element of an HTML page
- The <head> element contains meta information about the HTML page
- The <title> element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab)
- The <body> element defines the document's body, and is a container for all the visible contents, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.
- The <h1> element defines a large heading
- The <p> element defines a paragraph

A Simple HTML Document

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
```

```
<h1>My First Heading</h1>
<p>My first paragraph.</p>
<a href="https://www.w3schools.com">Visit W3Schools</a>

</body>
</html>
```

HTML Tags

As told earlier, HTML is a markup language and makes use of various tags to format the content. These tags are enclosed within angle braces **<Tag Name>**. Except few tags, most of the tags have their corresponding closing tags. For example, **<html>** has its closing tag **</html>** and **<body>** tag has its closing tag **</body>** tag etc.

Above example of HTML document uses the following tags:

Tag	Description
<!DOCTYPE...>	This tag defines the document type and HTML version.
<html>	This tag encloses the complete HTML document and mainly comprises of document header which is represented by <head>...</head> and document body which is represented by <body>...</body> tags.
<head>	This tag represents the document's header which can keep other HTML tags like <title>, <link> etc.
<title>	The <title> tag is used inside the <head> tag to mention the document title.
<body>	This tag represents the document's body which keeps other HTML tags like <h1>, <div>, <p> etc.
<h1>	This tag represents the heading.

Heading Tags Any document starts with a heading. You can use different sizes for your headings. HTML also has six levels of headings, which use the elements <h1>, <h2>, <h3>, <h4>, <h5>, <h6>

. While displaying any heading, browser adds one line before and one line after that heading.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Heading Example</title>
</head>
<body>
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>
<h5>This is heading 5</h5>
<h6>This is heading 6</h6>
</body>
</html>
```

This is heading 1

This is heading 2

This is heading 3

This is heading 4

This is heading 5

This is heading 6

Paragraph Tag

The **<p>** tag offers a way to structure your text into different paragraphs. Each paragraph of text should go in between an opening **<p>** and a closing **</p>** tag as shown below in the example:

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Paragraph Example</title>
</head>
<body>
<p>Here is a first paragraph of text.</p>
<p>Here is a second paragraph of text.</p>
<p>Here is a third paragraph of text.</p>
</body>
</html>
```

This will produce the following result:

```
Here is a first paragraph of text.
Here is a second paragraph of text.
Here is a third paragraph of text.
```

Line Break Tag

Whenever you use the element **
**, anything following it starts from the next line. This tag is an example of an empty element, where you do not need opening and closing tags, as there is nothing to go in between them.

The tag **
** has a space between the characters **br** and the forward slash. If you omit this space, older browsers will have trouble rendering the line break, while if you miss the forward slash character and just use **
** it is not valid in XHTML.

The href Attribute

The **<a>** tag defines a hyperlink. The **href** attribute specifies the URL of the page the link goes to:

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

The src Attribute

The **** tag is used to embed an image in an HTML page. The **src** attribute specifies the path to the image to be displayed:

```

```

The width and height Attributes

The tag should also contain the width and height attributes, which specify the width and height of the image (in pixels):

```

```

HTML <style> Tag

Definition and Usage

Use of the <style> element to apply a simple style sheet to an HTML document:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
h1 {color:red;}
```

```
p {color:blue;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

```
</body>
```

```
</html>
```

Output:

This is a heading

This is a paragraph.

What is CSS?

Cascading Style Sheets (CSS) is used to format the layout of a webpage.

With CSS, you can control the color, font, the size of text, the spacing between elements, how elements are positioned and laid out, what background images or background colors are to be used, different displays for different devices and screen sizes, and much more!

Using CSS

CSS can be added to HTML documents in 3 ways:

- **Inline** - by using the `style` attribute inside HTML elements
- **Internal** - by using a `<style>` element in the `<head>` section
- **External** - by using a `<link>` element to link to an external CSS file

Inline CSS

An inline CSS is used to apply a unique style to a single HTML element.

An inline CSS uses the `style` attribute of an HTML element.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1 style="color:blue; font-family:commanders; background-color:yellow;">A Blue Heading</h1>
```

```
<p style="color:red;">A red paragraph.</p>
</body>
</html>
```

Internal CSS

An internal CSS is used to define a style for a single HTML page.

An internal CSS is defined in the <head> section of an HTML page, within a <style> element.

The <style> tag is used to define style information (CSS) for a document.

Inside the <style> element you specify how HTML elements should render in a browser.

```
<html>
<head>
<style>
  h1 {color:red;}
  p {color:blue;}
</style>
</head>
<body>

<h1>A heading</h1>
<p>A paragraph.</p>

</body>
</html>
```

Example

```
<!DOCTYPE html>
```

```
<html>

<head>

<style>

.mydiv {

    border: 5px outset red;

    background-color: lightblue;

    text-align: center;

}

</style>

</head>

<body>

<h1>The div element</h1>

<div class="mydiv">

    <h2>This is a heading in a div element</h2>

    <p>This is some text in a div element.</p>

</div>

<p>This is some text outside the div element.</p>

</body>

</html>
```

External CSS

An external style sheet is used to define the style for many HTML pages.

To use an external style sheet, add a link to it in the `<head>` section of each HTML page:

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

The external style sheet can be written in any text editor. The file must not contain any HTML code, and must be saved with a .css extension.

styles.css:

```
body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

HTML – Iframes

HTML `<frame>` tag (Not supported in HTML5)

A `<frame>` tag is used with `<frameset>`, and it divides a webpage into multiple sections or frames, and each frame can contain different web pages.

it can appear anywhere in your document. The <iframe> tag defines a rectangular region within the document in which the browser can display a separate document, including scrollbars and borders. An inline frame is used to embed another document within the current HTML document.

```
<!DOCTYPE html>
<html>

  <head>
    <title>HTML Iframes</title>
  </head>

  <body>
    <p>Document content goes here...</p>

    <iframe src = "/html/menu.htm" width = "555" height = "200">
      Sorry your browser does not support inline frames.
    </iframe>

    <p>Document content also go here...</p>
  </body>

</html>
```

JavaScript

JavaScript became a fully independent language with its own specification called [ECMAScript](#), and now it has no relation to Java at all.

JavaScript is the world's most popular programming language.

JavaScript is the programming language of the Web.

JavaScript is easy to learn.

What makes JavaScript unique?

There are at least *three* great things about JavaScript:

- Full integration with HTML/CSS.
- Simple things are done simply.

- Supported by all major browsers and enabled by default.

JavaScript is the only browser technology that combines these three things

The **alert()** method displays an **alert** box with a message and an OK button.

A variable

A variable is a “named storage” for data. We can use variables to store goodies, visitors, and other data.

To create a variable in JavaScript, use the `let` keyword.

```
<!DOCTYPE html>
```

```
<script>
```

```
let message;
```

```
message = 'Hello!';
```

```
alert(message);
```

```
</script>
```

o/p

Hello

```
<!DOCTYPE html>
```

```
<script>
```

```
let b = 1; // define the variable and assign the value
```

```
let a=2;
```

```
alert(b+a); // Hello!
```

```
</script>
```

o/p

3

prompt

The function prompt accepts two arguments:

```
result = prompt(title, [default]);
```

It shows a modal window with a text message, an input field for the visitor, and the buttons OK/Cancel.

title

The text to show the visitor.

default

An optional second parameter, the initial value for the input field.

```
<!DOCTYPE html>
```

```
<script>
```

```
let age = prompt('How old are you?', 100);
```

```
alert(`You are ${age} years old!`); // You are 100 years old!
```

```
</script>
```

An embedded page at run.plnkr.co says

How old are you?



confirm

The syntax:

```
result = confirm(question);
```

The function confirm shows a modal window with a question and two buttons: OK and Cancel.

The result is true if OK is pressed and false otherwise.

```
<!DOCTYPE html>
<script>
```

```
let isBoss = confirm("Are you the boss?");
alert( isBoss ); // true if OK is pressed
</script>
```

O/P:

Are you the boss?

If ok click

True

Are you the boss?

If cancel click

false

Java Script With while:

```
<!DOCTYPE html>
<script>
```

```
let i = 0;
while (i < 3) {
  alert( i );      O/P: // shows 0, then 1, then 2
```

```
i++;  
}  
</script>
```

Java Script With Do While:

```
<!DOCTYPE html>  
<script>  
  
let i = 0;  
do {  
    alert( i );    O/P: // shows 0, then 1, then 2  
    i++;  
} while (i < 3);  
</script>
```

Java Script With If else:

```
<html>  
<body>  
<script>  
var a=20;  
if(a%2==0){  
    document.write("a is even number");    O/P: a is even number  
}  
else{  
    document.write("a is odd number");  
}  
</script>
```

Addition of Two Nos:

```
</script>  
</body>  
</html>  
<!DOCTYPE html>  
<html>  
<body>  
<h1>JavaScript Variables</h1>  
<p> x, y, and z are undeclared</p>  
<p>automatically declared when first used</p>
```

```
<p id="demo"></p>
<script>
x = 5;
y = 6;
z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
alert(z);
</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Statements</h2>
<p>A <b>JavaScript program</b> is a list of <b>statements</b> to be executed
by a computer.</p>
```

```
<p id="demo"></p>

<script>
var x, y, z;    // Declare 3 variables
x = 5;          // Assign the value 5 to x
y = 6;          // Assign the value 6 to y
z = x + y;      // Assign the sum of x and y to z
document.getElementById("demo").innerHTML =
"The value of z is " + z + ".";
</script>
</body>
</html>
```

Objects

An object can be created with figure brackets {...} with an optional list of *properties*. A property is a “key: value” pair, where key is a string (also called a “property name”), and value can be anything.

Syntax

```
let user = {    // an object
  name: "John", // by key "name" store value "John"
  age: 30       // by key "age" store value 30
};
```

Example

```
<!DOCTYPE html>
<script>
```

```
let user = {
  name: "John",
  age: 30
};
```

```
let key = prompt("What do you want to know about the user?", "name", );
```

```
// access by variable
alert( user[key] ); // John (if enter "name")
</script>
```

Example 2

```
<!DOCTYPE html>
<script>
"use strict";
```

```
function makeUser(name, age) {
  return {
    name: name,
    age: age,
    // ...other properties
  };
}
```

```
let user = makeUser("John", 30);
alert(user.name); // John
alert(user.age);
</script>
```

The **getElementById()** method is one of the most common methods in the HTML DOM. It is used almost every time you want to read or edit an HTML element

Changing HTML Content

The easiest way to modify the content of an HTML element is by using the **innerHTML** property.

To change the content of an HTML element, use this syntax:

```
document.getElementById(id).innerHTML = new HTML
```

Example:

```
<!DOCTYPE html>

<html>

<body>

<h2>My First JavaScript</h2>

<button type="button"
onclick="document.getElementById('demo').innerHTML = Date()">
Click me to display Date and Time.</button>

<p id="demo"></p>

</body>

</html>
```

Example:

```
<!DOCTYPE html>

<html>

<body>
```

```
<h2>JavaScript HTML DOM</h2>
<div id="main">
<p>Finding HTML Elements by Tag Name</p>
<p>This example demonstrates the <b>getElementsByName</b>
method.</p>
<p>hi hello</p>
</div>
<p id="demo"></p>
<script>
const x = document.getElementById("main");
const y = x.getElementsByTagName("p");
document.getElementById("demo").innerHTML =
"The first paragraph (index 0) inside "main" is: ' + y[2].innerHTML;
</script>
</body>
</html>
```

Output

JavaScript HTML DOM

Finding HTML Elements by Tag Name

This example demonstrates the **getElementsByName** method.

hi hello

The first paragraph (index 0) inside "main" is: hi hello

JavaScript Can Change HTML Content

One of many JavaScript HTML methods is **getElementById()**.

The example below "finds" an HTML element (with id="demo"), and changes the element content (innerHTML) to "Hello JavaScript":

JavaScript accepts both double and single quotes:

```
document.getElementById("demo").innerHTML = "Hello JavaScript";
```

```
document.getElementById('demo').innerHTML = 'Hello JavaScript';
```

```
<!DOCTYPE html>
<html>

<head>
  <title>
    Add JavaScript Code inside Head Section
  </title>

  <script>
    function myFun() {
      document.getElementById("demo")
        .innerHTML = "Content changed!";
    }
  </script>
</head>

<body>
  <h2>
    Add JavaScript Code
    inside Head Section
  </h2>

  <h3 id="demo" style="color:green;">
    Data Science
  </h3>

  <button type="button" onclick="myFun()">
    Click Here
  </button>
</body>

</html>
```

JavaScript Can Change HTML Attribute Values

```
<!DOCTYPE html>
<html>
<body>

<h2>What Can JavaScript Do?</h2>

<p>JavaScript can change HTML attribute values.</p>

<p>In this case JavaScript changes the value of the src (source) attribute of an
image.</p>

<button
onclick="document.getElementById('myImage').src='pic_bulbon.gif'">Turn
on the light</button>



<button
onclick="document.getElementById('myImage').src='pic_bulboff.gif'">Turn
off the light</button>

</body>
</html>
```

JavaScript Regular Expression

What Is a Regular Expression?

A regular expression is a sequence of characters that forms a **search pattern**.

The search pattern can be used for text search and text replace operations.

When you search for data in a text, you can use this search pattern to describe what you are searching for.

A regular expression can be a single character, or a more complicated pattern.

Regular expressions can be used to perform all types of **text search** and **text replace** operations.

A regular expression is a **pattern** of characters.

Syntax

/pattern/modifiers;

regular expressions are often used with the two **string methods**: **search()** and **replace()**.

The **search()** method uses an expression to search for a match, and returns the position of the match.

The **replace()** method returns a modified string where the pattern is replaced.

String search() With a Regular Expression

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1> String Methods</h1>
```

```
<p>Search a string for "VELS", and display the position of the match:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let text = "Visit VELs!";
```

```
let n = text.search(/VELS/i);
document.getElementById("demo").innerHTML = n;
</script>
</body>
</html>
```

Output:

String Methods

Search a string for "VELS", and display the position of the match:

6

Use String replace() With a Regular Expression

The **replace()** method replaces a specified value with another value in a string:

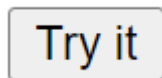
```
<!DOCTYPE html>
<html>
<body>
<h1> Replace Methods</h1>
<p>Replace "Microsoft" with "Data Science" in the paragraph
below:</p>
<button onclick="myFunction()">Try it</button>
<p id="demo">Please visit VELS!</p>
<script>
function myFunction() {
  let text = document.getElementById("demo").innerHTML;
  document.getElementById("demo").innerHTML =
    text.replace(/VELS/i,"DATA SCIENCE");
}
</script>
</body>
```

</html>

OUTPUT:

Replace Methods

Replace "Microsoft" with "Data Science" in the paragraph below:



Please visit VELS!

When you click on try it button

Please visit DATA SCIENCE!

Using test()

The **test()** method is a RegExp expression method.

It searches a string for a pattern, and returns true or false, depending on the result.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Regular Expressions</h2>
```

```
<p>Search for an "e" in the paragraph:</p>
```

```
<p id="p01">The Good things in life are free!</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let text = document.getElementById("p01").innerHTML;
```

```
const pattern = /Free!/i;
```

```
document.getElementById("demo").innerHTML = pattern.test(text);  
</script>  
</body>  
</html>
```

OUTPUT:

JavaScript Regular Expressions

Search for an "e" in the paragraph:

The Good things in life are free!

true

Modifiers

Modifiers are used to perform case-insensitive and global searches:

Modifier	Description
g	Perform a global match (find all matches rather than stopping after the first match)
i	Perform case-insensitive matching
m	Perform multiline matching

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
<h2> Regular Expressions</h2>
<p>case-insensitive search for "Vels" in a string:</p>
<p id="demo"></p>
<script>
let text = "Visit VELs";
let pattern = /Vels/i;
let result = text.match(pattern);
document.getElementById("demo").innerHTML = result;
</script>
</body>
</html>
```

OUTPUT:

Regular Expressions

case-insensitive search for "Vels" in a string:

VELS

JavaScript Events

The change in the state of an object is known as an **Event**. In html, there are various events which represents that some activity is performed by the user or by the browser. When javascript code is included in HTML, js react over these events and allow the execution. This process of reacting over the events is called **Event Handling**. Thus, js handles the HTML events via **Event Handlers**.

Event Performed	Event Handler	Description
click	onclick	When mouse click on an element

mouseover	onmouseover	When the cursor of the mouse comes over the element
mouseout	onmouseout	When the cursor of the mouse leaves an element
mousedown	onmousedown	When the mouse button is pressed over the element
mouseup	onmouseup	When the mouse button is released over the element
mousemove	onmousemove	When the mouse movement takes place.

Keyboard events:

Event Performed		Event Handler		Description
Keydown	&	onkeydown	&	When the user press and then release the key
Keyup		onkeyup		

Onclick:

```
<html>
```

```
<head> Javascript Events </head>
```

```
<script >
```

```
    function clickevent()
```

```
    {
```

```
        document.write("hi");
```

```
    }
```

```
</script>
```

```
<button onclick="clickevent()" >Who's this?</button>
```

```
</html>
```

MouseOver Event:

```
<html>
```

```
<head>
```

```
<h1> Javascript Events </h1>
```

```
</head>
```

```
<body>
```

```
<script>
```

```
    function mouseoverevent()  
    {  
        alert("This is VELS");  
    }
```

```
</script>
```

```
<p onmouseover="mouseoverevent()"> Keep cursor over me</p>
```

```
</body>
```

```
</html>
```

OUTPUT:



Focus Event

```
<html>

<head> Javascript Events</head>

<body>

<h2> Enter something here</h2>

<input type="text" id="input1" onfocus="focusevent()"/>

<script>

    function focusevent()

    {

        document.getElementById("input1").style.background=" blue";

    }

</script>

</body>

</html>
```

OUTPUT:

Javascript Events

Enter something here

Keydown Event

```
<html>

<head> Javascript Events</head>

<body>

<h2> Enter something here</h2>

<input type="text" id="input1" onkeydown="keydownevent()" />

<script>

    function keydownevent()

    {

        document.getElementById("input1");

        alert("Pressed a key");

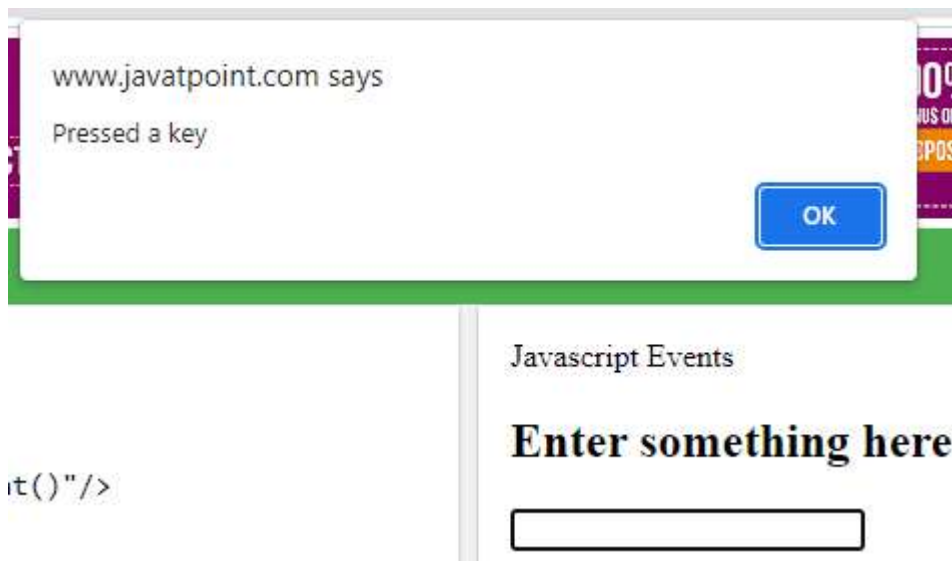
    }

</script>

</body>

</html>
```

OUTPUT:



Load event

```
<html>
<head>Javascript Events</head>
</br>
<p onload="window.alert('Page successfully loaded');"></p>
<script>
<!--
document.write("The page is loaded successfully");
//-->
</script>

</html>
```

VALIDATION and EVENT HANDLING IN JAVASCRIPT

Form validation occurs at the server, after the client had entered all necessary data and then pressed the Submit button. If some of the data that had been entered by the client had been in the wrong form or was simply missing, the server would have to send all the data back to the client and request that the form be resubmitted with correct information. This is a lengthy process and over burdening server. JavaScript, provides a way to validate form's data on the client's computer before sending it to the web server. Form validation generally performs two functions: Basic Validation - Checking the form to make sure that the data entered is right. Data Format Validation – Checking the data entered in the form for right value

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<script type="text/javascript">
```

```
function matchpass(){
```

```
var firstpassword=document.f1.password.value;
```

```
var secondpassword=document.f1.password2.value;
```

```
if(firstpassword==secondpassword){
```

```
return true;
```

```
}
```

```
else{
```

```
alert("password must be same!");
```

```
return false;
```

```
}
```

```
}
```

```
</script>
```

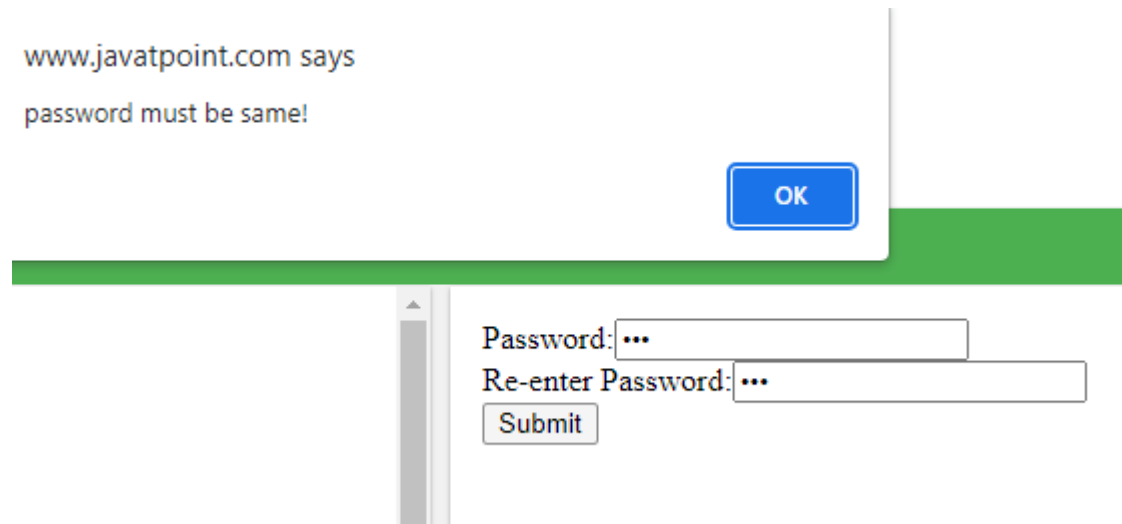
```
<form name="f1" action="http://www.javatpoint.com/javascriptpages/valid.jsp"  
onsubmit="return matchpass()">
```

```

Password:<input type="password" name="password" /><br/>
Re-enter Password:<input type="password" name="password2"/><br/>-
<input type="SUBMIT" >
</form>
</body></html>

```

OUTPUT:



```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation</h2>

<p>Please input a number between 1 and 10:</p>

<input id="numb">

<button type="button" onclick="myFunction()">Submit</button>

<p id="demo"></p>

```

```

<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  // If x is Not a Number or less than one or greater than 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>

</body>
</html>

```

OUTPUT:

JavaScript Validation

Please input a number between 1 and 10:

Input OK

Example:3

```

<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Validation</h2>
<p>Please input a number between 1 and 10:</p>
<input id="numb">
<input id="numb1">
<button type="button" onclick="myFunction()">Submit</button>
<p id="demo"></p>

```

```

<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  let y = document.getElementById("numb1").value;
  const z=parseInt(x)+parseInt(y);
  alert(z);
  document.getElementById("demo").innerHTML = z;

}
</script>
</body>
</html>

```

OUTPUT:

www.w3schools.com says

4

OK

JavaScript Validation

Please input a number between 1 and 10:

2	2	Submit
---	---	--------

4

Example:2

```

<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}

```

```
}  
}  
</script>  
</head>  
<body>  
  
<h2>JavaScript Validation</h2>  
  
<form name="myForm" action="/action_page.php" onsubmit="return  
validateForm()" method="post">  
  Name: <input type="text" name="fname">  
  <input type="submit" value="Submit">  
</form>  
  
</body>  
</html>
```

OUTPUT:

JavaScript Validation

Name:

After Submitting form:

Submitted Form Data

Your input was received as:

fname=packia

The server has processed your input and returned this answer.

Note: This tutorial will not teach you how servers are processing input. Processing input is explained in our [PHP tutorial](#).

Java Script Exception Handling:

What is Exception Handling

In programming, exception handling is a process or method used for handling the abnormal statements in the code and executing them. It also enables to handle the flow control of the code/program. For handling the code, various handlers are used that process the exception and execute the code. **For example**, the Division of a non-zero value with zero will result into infinity always, and it is an exception. Thus, with the help of exception handling, it can be executed and handled.

In exception handling:

A throw statement is used to raise an exception. It means when an abnormal condition occurs, an exception is thrown using throw.

Types of Errors

While coding, there can be three types of errors in the code:

1. **Syntax Error:** When a user makes a mistake in the pre-defined syntax of a programming language, a syntax error may appear.
2. **Runtime Error:** When an error occurs during the execution of the program, such an error is known as Runtime error. The codes which create runtime errors are known as Exceptions. Thus, exception handlers are used for handling runtime errors.
3. **Logical Error:** An error which occurs when there is any logical mistake in the program that may not produce the desired output, and may terminate abnormally. Such an error is known as Logical error.

Throw/, and Try...Catch...Finally

The **try** statement defines a code block to run (to try).

The **catch** statement defines a code block to handle any error.

The **finally** statement defines a code block to run regardless of the result.

The **throw** statement defines a custom error.

Errors Will Happen!

When executing JavaScript code, different errors can occur.

Errors can be coding errors made by the programmer, errors due to wrong input, and other unforeseeable things.

Example

In this example we misspelled "alert" as "adddlert" to deliberately produce an error:

```
<!DOCTYPE html>

<html>

<body>

<h2>JavaScript Error Handling</h2>

<p>How to use <b>catch</b> to display an error.</p>

<p id="demo"></p>

<script>

try {

    adddlert("Welcome guest!");

}

catch(err) {

    document.getElementById("demo").innerHTML = err.message;

}
```

```
</script>
```

```
</body>
```

```
</html>
```

try...catch example

```
<html>
```

```
<head> Exception Handling</br></head>
```

```
<body>
```

```
<script>
```

```
try{
```

```
var a= ["34","32","5","31","24","44","67"]; //a is an array
```

```
document.write(a);    // displays elements of a
```

```
document.write(b);    //b is undefined but still trying to fetch its value. Thus  
catch block will be invoked
```

```
}
```

```
catch(e)
```

```
{
```

```
alert("There is error which shows "+e.message); //Handling error
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Output:

Exception Handling
34,32,5,31,24,44,67

www.javatpoint.com says

There is error which shows b is not defined



Object Cloning in Java:

The **object cloning** is a way to create exact copy of an object. The clone() method of Object class is used to clone an object.

The **java.lang.Cloneable interface** must be implemented by the class whose object clone we want to create. If we don't implement Cloneable interface, clone() method generates **CloneNotSupportedException**.

The **clone() method** is defined in the Object class. Syntax of the clone() method is as follows:

protected Object clone() **throws** CloneNotSupportedException

Why use clone() method ?

The **clone() method** saves the extra processing task for creating the exact copy of an object. If we perform it by using the new keyword, it will take a lot of processing time to be performed that is why we use object cloning.

Advantage of Object cloning

Although Object.clone() has some design issues but it is still a popular and easy way of copying objects.

It is the easiest and most efficient way for copying objects, especially if we are applying it to an already developed or an old project. Just define a parent class, implement Cloneable in it, provide the definition of the clone() method and the task will be done.

- Clone() is the fastest way to copy array.

I)Using Assignment Operator to create a copy of the Object

```
import java.io.*;

class Test {
    int x, y;
    Test()
    {
        x = 10;
        y = 20;
    }
}

class Main {
    public static void main(String[] args)
    {
        Test ob1 = new Test();
        System.out.println(ob1.x + " " + ob1.y);

        Test ob2 = ob1;

        ob2.x = 100;

        System.out.println(ob1.x + " " + ob1.y);
        System.out.println(ob2.x + " " + ob2.y);
    }
}
```

Output:

```
10 20
100 20
100 20
```

II.Creating a copy using the clone() method

```
class Stu implements Cloneable{

    int rollno;
```

```
String name;

Stu(int Rno,String stuname){

rollno=Rno;

name=stuname;

}

public Object clone()throws CloneNotSupportedException{

return super.clone();

}

public static void main(String args[]){

try{

Stu s1=new Stu(100,"amma");

Stu s2=(Stu)s1.clone();

System.out.println(s1.rollno+" "+s1.name);

System.out.println(s2.rollno+" "+s2.name);

}catch(CloneNotSupportedException c){}

}

}
```

Output:

```
100 amma
100 amma
```